Machine Learning-Based Scandinavian Hotel Price Prediction and Development of Smart Web Application

Çağdaş Can Azim
Department of Computer Engineering
Biruni University
230404012@st.biruni.edu.tr

Abstract

Hotel price prediction is an important problem in the tourism and hospitality industry because accommodation prices are affected by many different factors such as country, city, season, hotel score, review count, distance to city center, star rating, and available amenities. In this study, a machine learning-based hotel price prediction and classification system was developed using an original hotel dataset collected by scraping hotel information from the Booking application. The collected dataset contains hotel records from Scandinavian countries and includes numerical, categorical, and binary amenity-based features. The original hotel price values were cleaned and converted into numerical Turkish Lira values. After this process, hotel price categories were created as Economy, Standard, Premium, and Luxury.

In the classification phase, five machine learning models and three deep learning models were trained and evaluated. These models include Logistic Regression, K-Nearest Neighbors, Support Vector Machine, Random Forest, Gradient Boosting, Basic Artificial Neural Network, Artificial Neural Network with Dropout, and Deep Artificial Neural Network with Batch Normalization. The models were evaluated using accuracy, precision, recall, F1-score, AUC, training time, and runtime for one record. Among the classification models, Random Forest achieved the best overall performance with 60.06% accuracy, 59.66% F1-score, and 83.66% AUC.

In addition to classification, a regression-based model was developed for the final web application. A Random Forest Regressor was trained to predict the numerical hotel price. The regression model achieved a Mean Absolute Error of 11,180.66 TL, Root Mean Squared Error of 17,913.12 TL, and R^2 score of 0.482. Finally, the trained regression model was integrated into a Streamlit web application. The application first predicts the estimated hotel price and then converts the predicted price into a category. The developed system provides a user-friendly output including estimated price, predicted category, and expected price range.

Keywords— Hotel price prediction, machine learning, deep learning, regression, classification, feature selection, web scraping, Streamlit, web application.

1. Introduction

Hotel prices vary depending on many different factors. A hotel located in a central area, having a high user score, offering more amenities, or being located in a popular destination may have a higher price than other hotels. Similarly, season, country, city, star rating, customer review count, and available hotel services may also affect accommodation prices. Therefore, predicting hotel prices is a useful task for both customers and businesses.

From the customer perspective, hotel price prediction can help users understand whether a hotel is economical, standard, premium, or luxury. From the business perspective, such models can support pricing analysis, market comparison, and decision-making. Since hotel prices are affected by multiple nonlinear factors, machine learning methods are suitable for this prediction problem.

In this project, a machine learning-based system was developed to predict hotel prices using an original dataset collected by scraping hotel information from the Booking application. The dataset consists of hotels located in Scandinavian countries. Unlike projects that use ready-made public datasets, this study includes an own dataset collection process. Therefore, the project covers both data collection and machine learning model development stages.

The project has two main modeling objectives. The first objective is to classify hotels into four price categories: Economy, Standard, Premium, and Luxury. For this purpose, several machine learning and deep learning classification models were trained and compared. The second objective is to predict the numerical hotel price and then convert this predicted price into a category. This second approach was used in the final Streamlit web application because it provides more informative output to the user.

2. Related Work

Machine learning has been widely used in prediction problems in many fields such as real estate, e-commerce, transportation, tourism, and finance. In price prediction tasks, models generally use structured variables such as location, rating, number of reviews, distance, product or service characteristics, and seasonal information. Hotel price prediction is similar to these problems because hotel prices are also influenced by many structured and categorical features.

Traditional statistical models can provide simple baseline predictions, but they may not be sufficient when relationships between features and prices are nonlinear. For this reason, machine learning models such as Random Forest, Gradient Boosting, Support Vector Machine, and K-Nearest Neighbors are frequently used in tabular prediction problems. Ensemble models are especially useful because they combine multiple decision trees and can capture complex interactions between features.

Deep learning models can also be applied to structured datasets. Artificial Neural Networks can learn nonlinear patterns, but they often require sufficient data and careful regularization. For tabular datasets, tree-based models may sometimes outperform deep learning models because they are more robust and easier to train on structured data.

In recent years, machine learning models have also been integrated into web applications to make prediction systems more accessible. Streamlit is one of the tools used for this purpose because it allows rapid development of interactive machine learning applications. In this project, Streamlit was used to deploy the trained hotel price prediction model and provide real-time predictions through a user-friendly interface.

3. Methodology

The overall methodology of this study consists of dataset collection, data cleaning, exploratory data analysis, preprocessing, feature engineering, model development, model evaluation, feature selection, and web application deployment. The proposed workflow starts with collecting hotel data from the Booking application and continues with transforming the dataset into a suitable format for machine learning models.

Unlike studies that use ready-made public datasets, this project uses an original dataset collected through scraping. Therefore, the project satisfies the own dataset collection requirement and demonstrates the complete machine learning pipeline from raw data collection to real-time web application deployment.

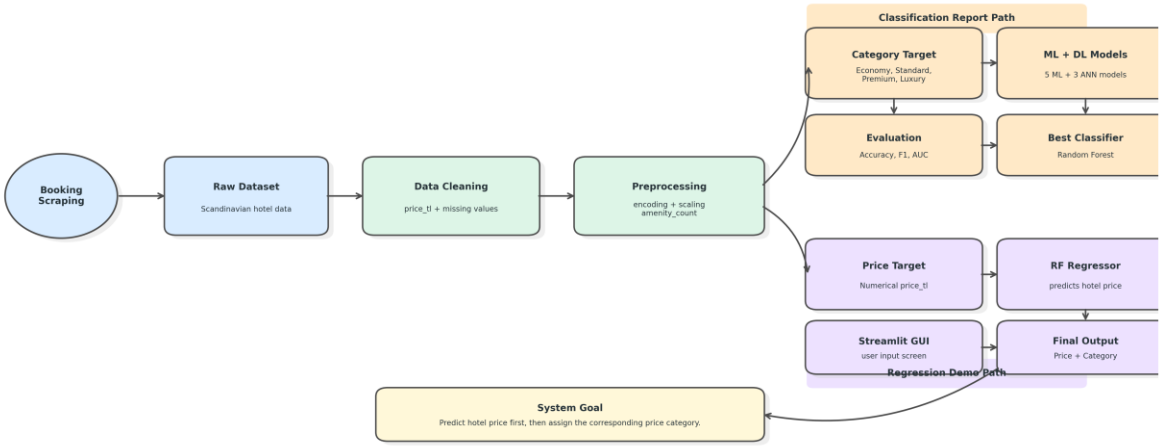


Figure 1: Workflow of the proposed hotel price prediction and smart web application system.

3.1 Data Collection

In this study, the dataset was collected by scraping hotel information from the Booking application. The aim of the data collection process was to create an original and real-world hotel dataset that could be used for hotel price prediction and price category classification. Therefore, the dataset was not taken directly from a public machine learning repository; instead, it was created specifically for this project.

The collected data consists of hotel records from Scandinavian countries. Each record includes information such as country, city, season, hotel score, review count, distance to city center, star rating, original price, and available hotel amenities. The dataset includes numerical, categorical, and binary variables. Numerical features include score, review count, distance to center, star rating, and amenity count. Categorical features include country, city, and season. Binary features represent whether a hotel provides a specific service or facility, such as Wi-Fi, parking, pool, spa, breakfast, restaurant, airport shuttle, elevator, kitchen, balcony, and similar amenities.

The original price column contained textual values such as Turkish Lira prices and hidden price values. For example, some price values appeared in formats such as “TL 84.723”, while some records contained hidden price information such as “Fiyat Gizli”. Therefore, the price information required a cleaning process before model training. Visible price values were converted into numerical Turkish Lira values, while hidden price values were treated as missing values and removed from supervised learning tasks.

Since the dataset was collected through scraping, it can be considered an original project dataset. This aspect is important because it demonstrates that the data collection process was performed as part of the study.

Table 1: Features of the Scandinavian Hotel Dataset with Typical Value Ranges

No.	Feature	Description	Range
01	country	Country where the hotel is located.	Categorical
02	city	City where the hotel is located.	Categorical
03	season	Season of booking or stay.	Spring, Summer, Autumn, Winter
04	score	User rating score of the hotel.	0.0–10.0
05	review_count	Number of user reviews for the hotel.	0–100000+
06	distance_to_center	Distance of the hotel from the city center.	0–50 km
07	star_rating	Official star rating of the hotel.	0–5
08	has_free_cancellation	Whether the hotel offers free cancellation.	0–1
09	has_wifi	Whether the hotel provides Wi-Fi service.	0–1
10	has_pool	Whether the hotel has a swimming pool.	0–1
11	has_parking	Whether the hotel provides parking service.	0–1
12	has_air_conditioning	Whether the hotel has air conditioning.	0–1
13	has_fitness	Whether the hotel has a fitness center.	0–1
14	has_spa	Whether the hotel provides spa service.	0–1
15	has_family_rooms	Whether the hotel has family rooms.	0–1
16	has_non_smoking	Whether the hotel has non-smoking rooms.	0–1
17	has_restaurant	Whether the hotel has a restaurant.	0–1
18	has_bar	Whether the hotel has a bar.	0–1
19	is_pet_friendly	Whether the hotel allows pets.	0–1
20	has_room_service	Whether the hotel provides room service.	0–1
21	is_wheelchair_accessible	Whether the hotel is wheelchair accessible.	0–1
22	has_balcony	Whether the hotel has balcony rooms.	0–1
23	has_airport_shuttle	Whether the hotel provides airport shuttle service.	0–1
24	has_breakfast	Whether the hotel offers breakfast.	0–1
25	has_24h_front_desk	Whether the hotel has a 24-hour front desk.	0–1
26	has_elevator	Whether the hotel has an elevator.	0–1
27	has_soundproofing	Whether the hotel has soundproof rooms.	0–1
28	is_beachfront	Whether the hotel is located on the beachfront.	0–1
29	has_kitchen	Whether the hotel room has a kitchen.	0–1
30	has_private_bathroom	Whether the hotel has private bathroom facilities.	0–1
31	has_ev_charging	Whether the hotel provides electric vehicle charging.	0–1
32	is_adults_only	Whether the hotel is adults only.	0–1
33	no_prepayment	Whether the hotel allows booking without prepayment.	0–1
34	is_remote_friendly	Whether the hotel is suitable for remote working.	0–1
35	amenity_count	Total number of available hotel amenities.	0–30+
36	price_tl	Cleaned hotel price in Turkish Lira.	0–200000+ TL
37	price_category	Generated hotel price category.	Ekonomik / Standart / Premium / Lüks

Table 1: Features of the Scandinavian Hotel Dataset wit h Typical Value Ranges.

3.2 Dataset Overview and Feature Analysis

After cleaning the price column, a new numerical target variable named price_tl was created. This variable represents the hotel price in Turkish Lira. In addition, the numerical price values were converted into four hotel price categories:

Price Range	Category
0 – 25,000 TL	Economy
25,001 – 40,000 TL	Standard
40,001 – 70,000 TL	Premium
70,001 TL and above	Luxury

These categories were used for the classification task. For the regression task, the original numerical price value was used as the target variable.

Exploratory data analysis was performed to understand the distribution of features and their relationship with the price categories. Boxplots were used to compare feature distributions across price categories. A correlation matrix was created to observe relationships between numerical and binary features. In addition, original and normalized feature distributions were visualized to understand skewness, spread, and potential outliers.

To examine the distribution of numerical and binary features across price categories, boxplot analysis was performed. The results are presented in Figure 2.

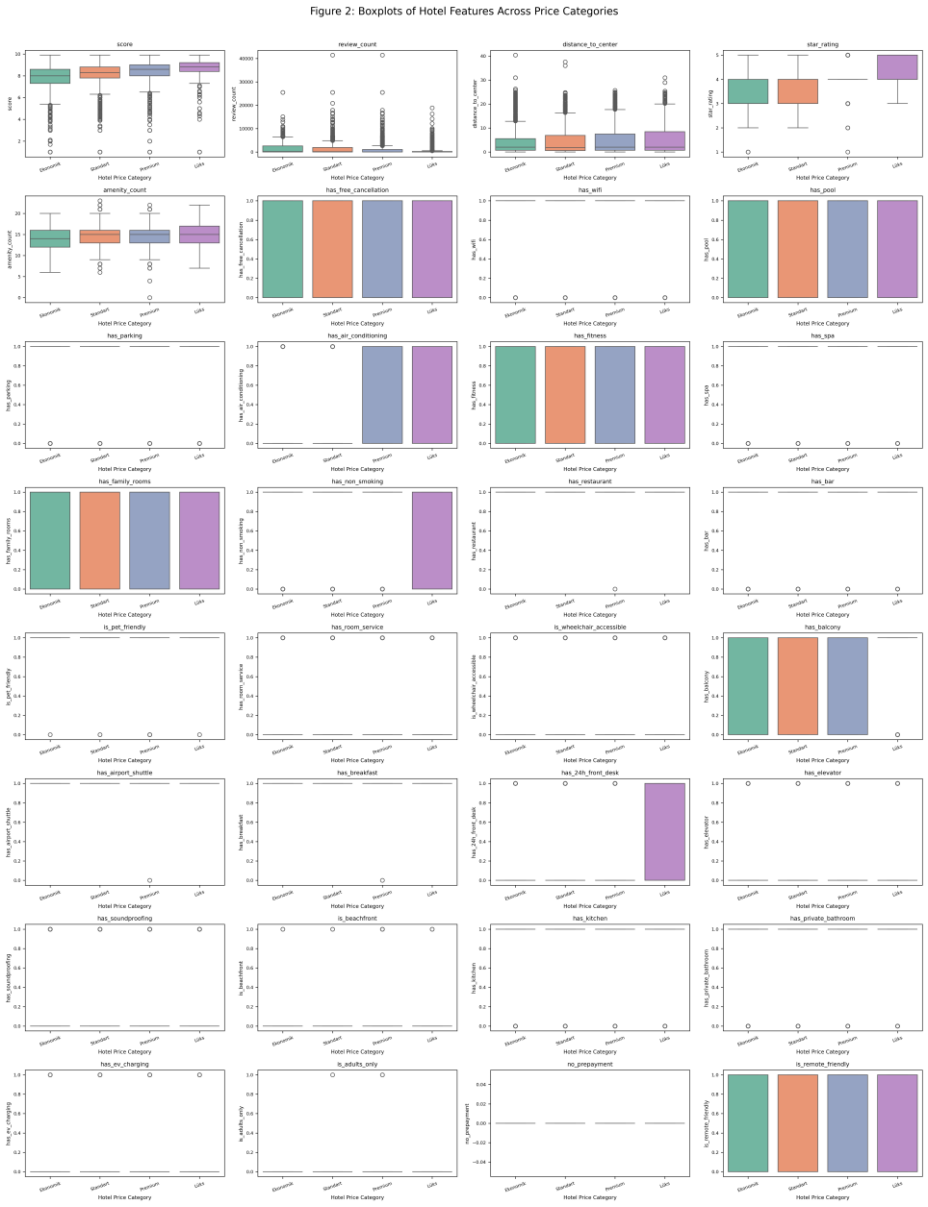


Figure 2: Boxplots showing the distribution and outliers of hotel features across the four price categories.

In addition to distribution analysis, a correlation matrix was generated to observe the relationships among numerical and binary hotel features. The correlation matrix is shown in Figure 3.

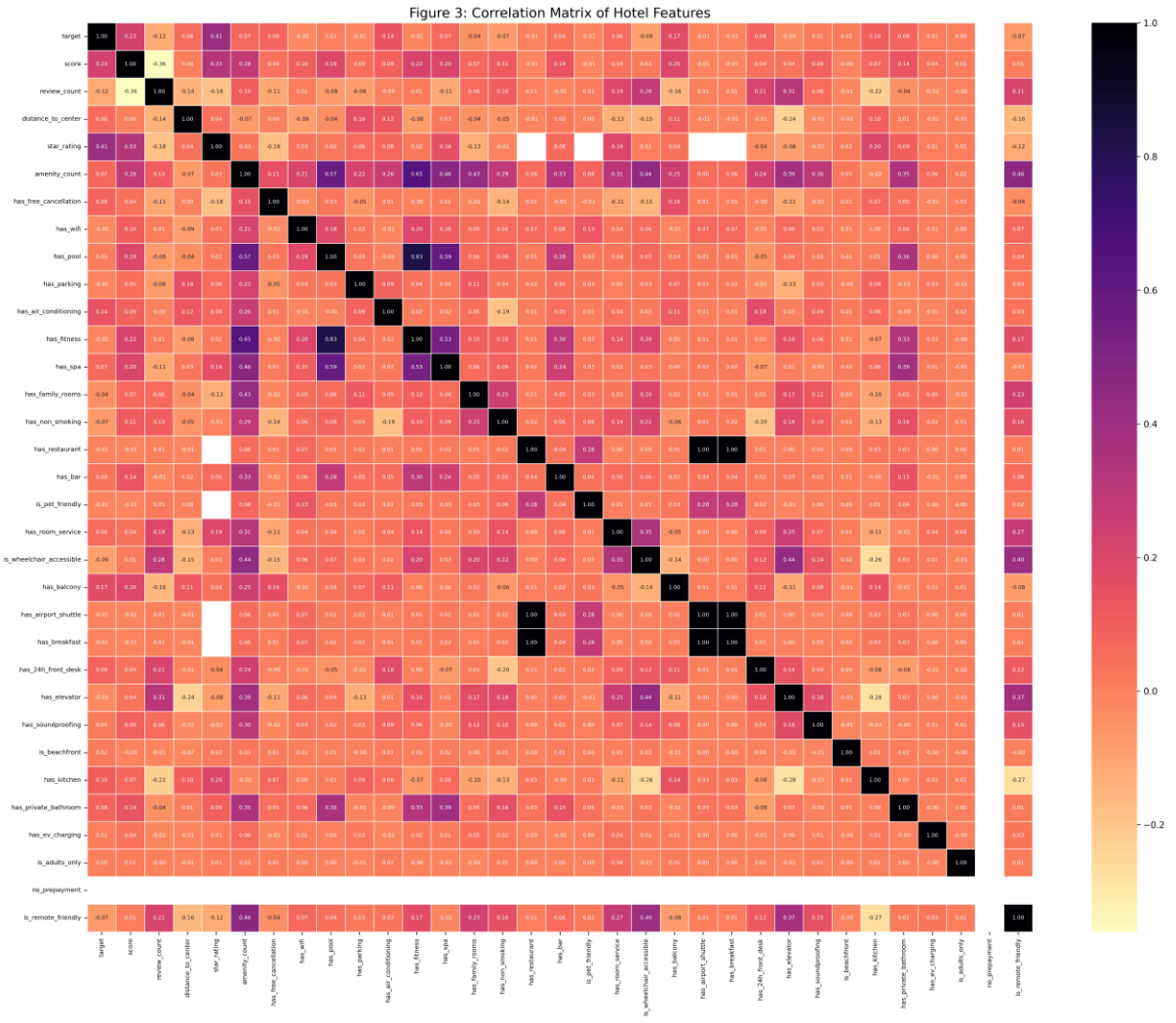


Figure 3: Correlation matrix of hotel features.

The original feature distributions before normalization were also visualized to observe skewness, spread, and possible outliers. These distributions are shown in Figure 4.

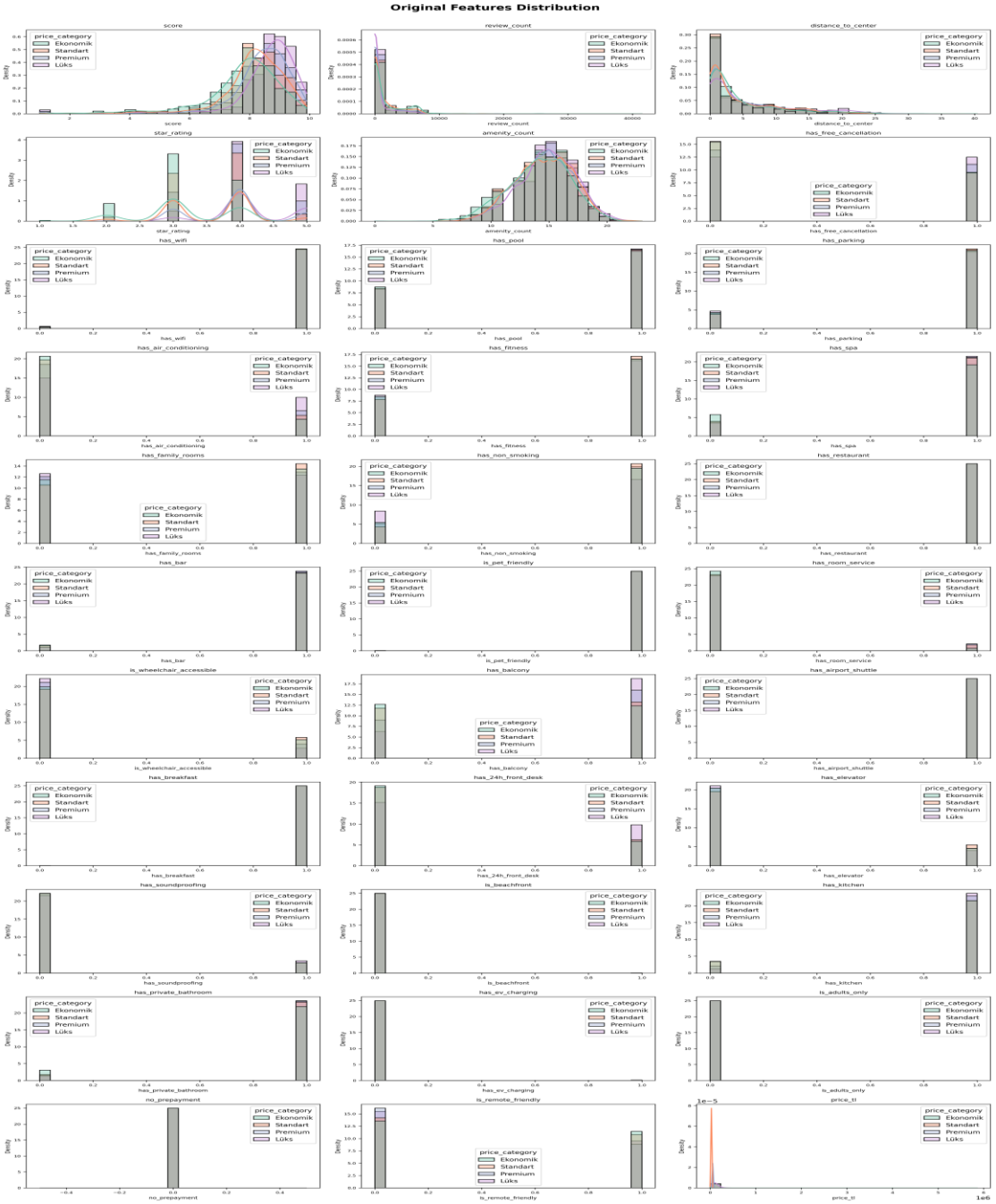


Figure 4: Original distribution of all features before normalization.

3.3 Data Preprocessing

Data preprocessing was one of the most important steps in this project because the dataset contained different types of variables. The preprocessing pipeline included price cleaning, missing value handling, encoding, feature scaling, and feature engineering.

First, the original `price` column was cleaned. Non-numeric characters were removed, and the remaining values were converted into integer Turkish Lira values. Hidden price values were removed from supervised learning tasks because they could not be used as target values.

Second, numerical features such as `score`, `review_count`, `distance_to_center`, and `star_rating` were converted into numerical format. Missing numerical values were handled using median imputation.

Third, categorical features such as `country`, `city`, and `season` were transformed using one-hot encoding. This was necessary because machine learning models cannot directly process textual categorical values.

Fourth, numerical variables were standardized using StandardScaler. This step is especially important for algorithms such as Logistic Regression, K-Nearest Neighbors, Support Vector Machine, and Artificial Neural Networks because these models are sensitive to feature scale.

Finally, an additional feature named `amenity_count` was created. This feature represents the total number of available hotel amenities. It was calculated by summing all binary amenity columns for each hotel.

After missing value handling and encoding, numerical features were standardized to reduce the effect of scale differences. The normalized feature distributions are shown in Figure 5.

Figure 5: Normalized Features Distribution

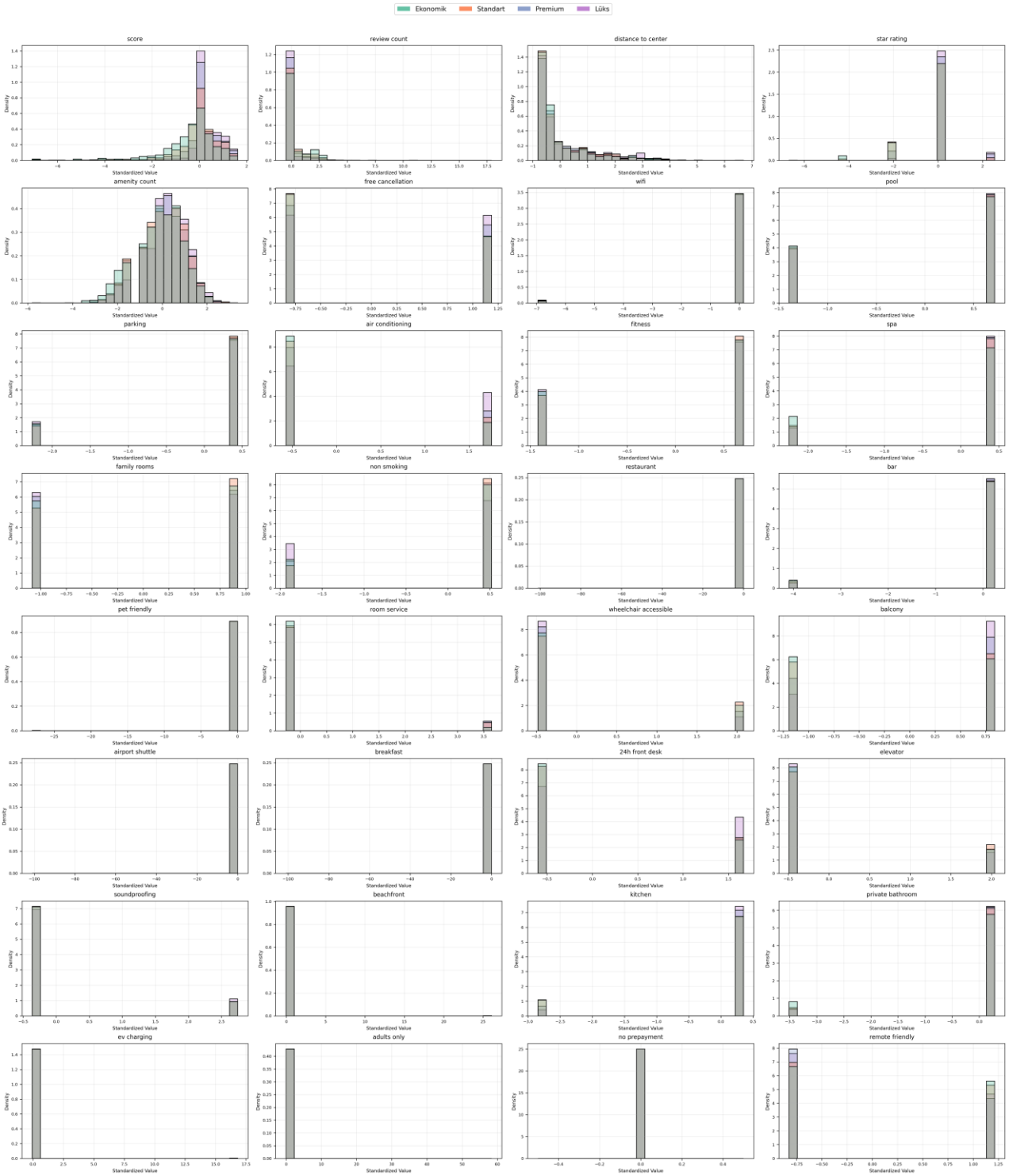


Figure 5: Normalized feature distributions after preprocessing and standardization.

3.4 Label Encoding and One-Hot Encoding

For classification, the hotel price categories were label encoded. The category mapping was defined as:

0 → Economy

1 → Standard

2 → Premium

3 → Luxury

The features `country`, `city`, and `season` were encoded using one-hot encoding. After one-hot encoding, the number of processed model input features increased. Therefore, the model did not use only the original raw columns; instead, it used the transformed feature matrix.

This is important for feature selection because feature selection was performed on the processed feature matrix, not directly on the raw dataset. For example, the raw `city` column was transformed into multiple binary columns representing different cities. Therefore, feature importance was calculated after preprocessing

3.5 Feature Normalization

Feature normalization was applied to numerical variables using StandardScaler. This method removes the mean and scales the data to unit variance. Normalization prevents variables with large numerical ranges from dominating the learning process.

For example, `review_count` may contain large values, while `score` is generally between 0 and 10. Without scaling, distance-based and gradient-based models may be biased toward features with larger numeric ranges. Therefore, scaling improved the consistency of model training.

3.6 Class Imbalance Handling

The price categories were not equally distributed in the dataset. Some categories had more samples than others. This class imbalance can cause models to become biased toward majority classes. For this reason, SMOTE was applied to the training data for class balancing.

SMOTE creates synthetic samples for minority classes and helps the model learn a more balanced decision boundary. In this project, SMOTE was applied only to the training set to avoid data leakage. The confusion matrix before and after SMOTE was compared for the best-performing machine learning model.

The purpose of this step was to observe whether class balancing improved the prediction performance, especially for minority price categories. The comparison showed how the model's prediction behavior changed after balancing the training data.

SMOTE was applied only to the training data to reduce class imbalance. The effect of SMOTE was evaluated by comparing the confusion matrices before and after balancing.

3.7 Feature Engineering and Feature Selection

Feature engineering was performed by creating the `amenity_count` feature. This feature summarizes the number of hotel services and facilities. Since hotels with more amenities may have higher prices, this feature was expected to contribute to prediction performance.

Feature selection was performed using Random Forest feature importance scores. Since categorical variables were transformed using one-hot encoding, the feature importance scores were calculated on the processed feature matrix. The all-feature model used 111 processed features. Then, the top 15 most important features were selected and used to retrain the Random Forest model.

This comparison was used to evaluate whether reducing the number of features could preserve model performance while simplifying the model.

Random Forest feature importance was used to determine the most informative processed features. Since one-hot encoding was applied to categorical variables, feature selection was performed on the transformed feature matrix. The top 20 important features are shown in Figure 7.

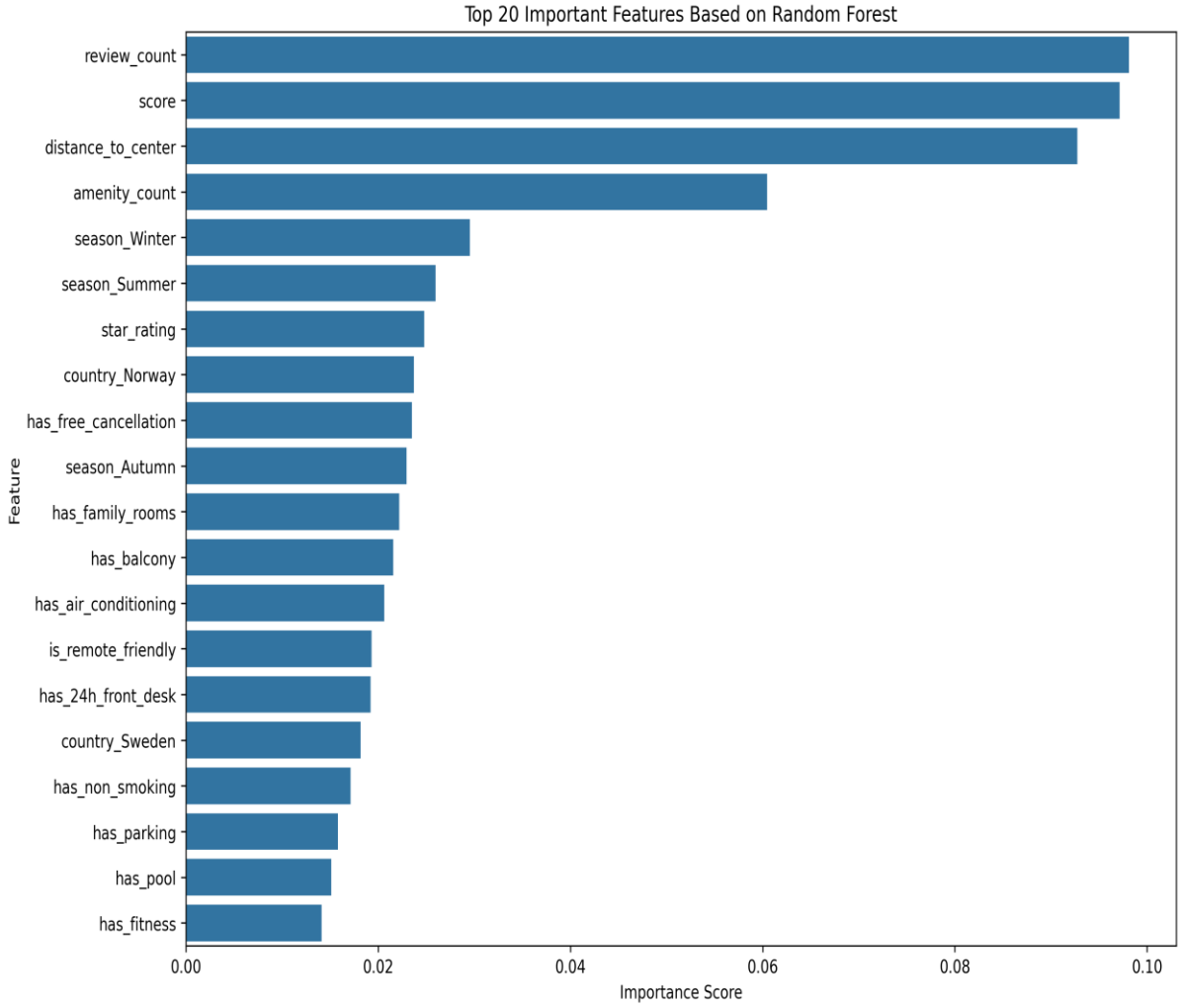


Figure 7: Top 20 most important features based on Random Forest feature importance.

3.8 Model Development

In this study, five machine learning models and three deep learning models were developed for hotel price category classification. The machine learning models were: Logistic Regression K-Nearest Neighbors, Support Vector Machine, Random Forest, Gradient Boosting.

The deep learning models were: Basic Artificial Neural Network, Artificial Neural Network with Dropout, Deep Artificial Neural Network with Batch Normalization.

For the final web application, a separate regression model was developed: Random Forest Regressor.

The classification models were used to compare category prediction performance. The regression model was used in the final Streamlit application to predict the numerical hotel price and then convert it into a category.

The machine learning and deep learning models used in this study, along with their main hyperparameters, are presented in Table 3.

Table 3: Machine Learning and Deep Learning Models with Tuned Hyperparameters

Sr No	Model	Tuning Parameters
1	Logistic Regression	max_iter=1000, C=1.0, solver='lbfgs', class_weight='balanced', random_state=42
2	K-Nearest Neighbors	n_neighbors=5, weights='distance', metric='minkowski', p=2
3	Support Vector Machine	C=10, kernel='rbf', gamma='scale', probability=True, class_weight='balanced', random_state=42
4	Random Forest	n_estimators=300, max_depth=20, min_samples_split=2, min_samples_leaf=1, class_weight='balanced', random_state=42
5	Gradient Boosting	n_estimators=200, learning_rate=0.05, max_depth=3, random_state=42
6	DL Basic ANN	Dense layers: 64-32, activation='relu', optimizer='adam', loss='categorical_crossentropy', epochs=50, batch_size=32
7	DL ANN with Dropout	Dense layers: 128-64-32, Dropout=0.30/0.20, activation='relu', optimizer='adam', epochs=70, batch_size=32
8	DL Deep ANN + BatchNorm	Dense layers: 256-128-64-32, BatchNormalization, Dropout=0.30/0.25/0.20, Adam learning_rate=0.001, epochs=100

Table 3: Machine learning and deep learning models used for hotel price category classification with tuned hyperparameters.

3.9 Model Evaluation

The classification models were evaluated using the following metrics:

Accuracy, Precision, Recall, F1-score, AUC, Training Time, Runtime for One Record

Accuracy measures the ratio of correct predictions to all predictions. Precision measures how many predicted positive samples were actually correct. Recall measures how many actual positive samples were correctly predicted. F1-score is the harmonic mean of precision and recall. AUC measures the model's ability to separate classes.

For the regression model, the following metrics were used:

Mean Absolute Error, Root Mean Squared Error, R^2 Score

Mean Absolute Error shows the average absolute prediction error in Turkish Lira. Root Mean Squared Error gives more penalty to larger errors. R^2 score shows how much of the variance in the target price is explained by the model.

The confusion matrix structure used for multi-class hotel price category classification is presented in Table 4.

Table 4: Confusion Matrix Structure

Unnamed: 0	Predicted: Ekonomik	Predicted: Standart	Predicted: Premium	Predicted: Lüks
Actual: Ekonomik	TP1	FP21	FP31	FP41
Actual: Standart	FN12	TP2	FP32	FP42
Actual: Premium	FN13	FN23	TP3	FP43
Actual: Lüks	FN14	FN24	FN34	TP4

Table 4: Confusion matrix structure for four-class hotel price category classification.

4. Applied Machine Learning and Deep Learning Techniques

4.1 Logistic Regression

Logistic Regression is a linear classification model. It estimates the probability of each class using a linear combination of input features. Although it is simple and interpretable, it may not perform well when the relationship between features and target classes is nonlinear. In this project, Logistic Regression was used as a baseline model.

$$P(y = i \mid \mathbf{x}) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

4.2 K-Nearest Neighbors

K-Nearest Neighbors is a distance-based learning algorithm. It classifies a new sample according to the majority class of its nearest neighbors. Since it depends on distance calculations, feature scaling is very important for KNN. In this project, KNN was applied after standardization.

$$d(x, x_i) = \sum_{j=1}^{n} (x_j - x_{i,j})^2$$

4.3 Support Vector Machine

Support Vector Machine aims to find an optimal decision boundary that separates classes with the maximum margin. The RBF kernel was used to capture nonlinear relationships between hotel features and price categories. SVM produced competitive results but required a longer training time compared to other models.

$$\min_{\mathbf{w},b} \frac{1}{2}\mathbf{w}^2 + C\sum_{i=1}^{n} \max\left(0, 1 - y_i(\mathbf{w}^T\mathbf{x}_i + b)\right).$$

$$\hat{y} = \operatorname{argmax}_k \left(\mathbf{w}_k^T\mathbf{x} + b_k\right)$$

4.4 Random Forest

Random Forest is an ensemble learning method that combines multiple decision trees. Each tree is trained on a different sample of the data, and the final prediction is obtained by combining the predictions of all trees. Random Forest is effective for tabular datasets because it can capture nonlinear relationships and feature interactions. In this project, Random Forest achieved the best classification performance.

$$\hat{y} = mode\{h_1(\mathbf{x}), h_2(\mathbf{x}), \ldots, h_T(\mathbf{x})\}$$

4.5 Gradient Boosting

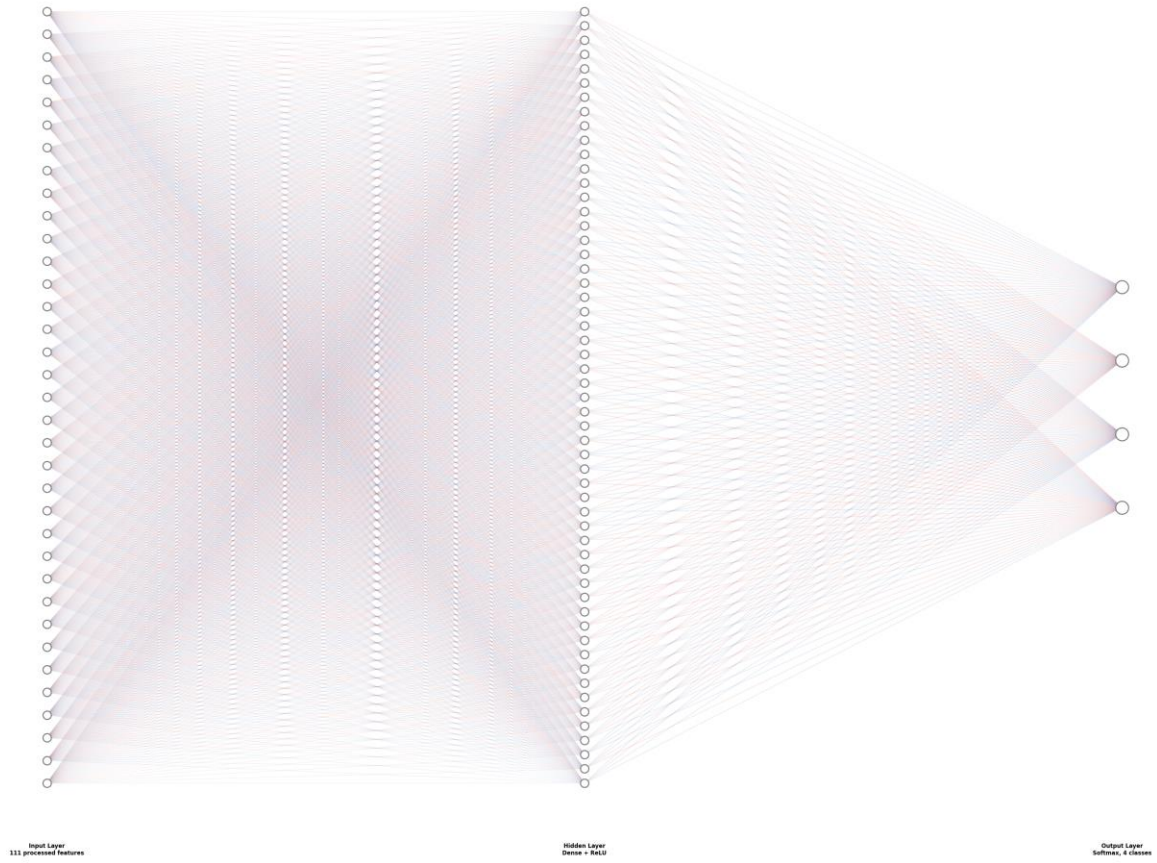
Gradient Boosting is another ensemble learning method. It builds trees sequentially, where each new tree attempts to correct the errors of previous trees. This allows the model to capture complex patterns in the data. In this project, Gradient Boosting was evaluated as one of the machine learning classification models.

$$F_M(x) = F_0(x) + \sum_{m=1}^{M} \eta \cdot h_m(x)$$

4.6 Artificial Neural Networks

Artificial Neural Networks were used to model nonlinear relationships between hotel features and price categories. Three neural network architectures were tested. The Basic ANN used dense layers with ReLU activation. The ANN with Dropout used dropout layers to reduce overfitting. The Deep ANN with Batch Normalization used a deeper architecture with batch normalization and dropout.

Although deep learning models performed reasonably, they did not outperform Random Forest in this dataset.



4.7 Random Forest Regressor

Random Forest Regressor was used for the final price prediction system. Unlike classification models, this model predicts a continuous numerical value. In this project, it predicted the hotel price in Turkish Lira.

Since hotel prices may have a skewed distribution, the target variable was transformed using `log1p` before training. After prediction, the output was converted back to Turkish Lira using `expm1`. Then, the predicted price was converted into a category using predefined price ranges.

5. Results and Discussion

5.1 Classification Results

The performance of eight classification models was compared. The initial evaluation results of all classification models are presented in Table 7.

Table 7: Initial Evaluation of Classification Models Before Cross-Validation

Model	Accuracy	Precision	Recall	F1-score	AUC	Training Time (se	Runtime for One Record (se
Random Forest	60.06	61.23	58.5	59.66	83.66	4.6835	7.665e-05
Support Vector Machine	54.63	53.58	57.61	54.72	80.68	54.6714	0.00102705
DL ANN with Dropout	53.79	54.01	52.45	53.08	78.89	22.8015	0.00012147
DL Basic ANN	51.74	52.12	50.78	51.36	77.84	14.7126	0.00012113
K-Nearest Neighbors	52.47	51.92	50.07	50.85	77.02	0.0042	0.00013872
Gradient Boosting	52.18	56.79	47.98	50.42	77.13	19.8893	1.226e-05
Logistic Regression	42.58	43.16	48.63	42.97	72.8	2.8517	1.56e-06
DL Deep ANN + BatchNorı	42.19	52.69	31.69	28.98	68.57	11.9481	0.00017746

Although the deeper ANN architecture used Batch Normalization and Dropout, it did not improve performance. This shows that a deeper model is not always better for tabular datasets, especially when the dataset size and feature structure are limited.

Overall, Random Forest was selected as the strongest classification model.

5.2 ROC Curve Analysis

The ROC curve comparison was performed using the one-vs-rest strategy for the four hotel price categories. The AUC values show how well each model can distinguish between classes.

The ROC curve comparison of the classification models is shown in Figure 11.

Figure 11: ROC Curve Comparison for Hotel Price Category Classification

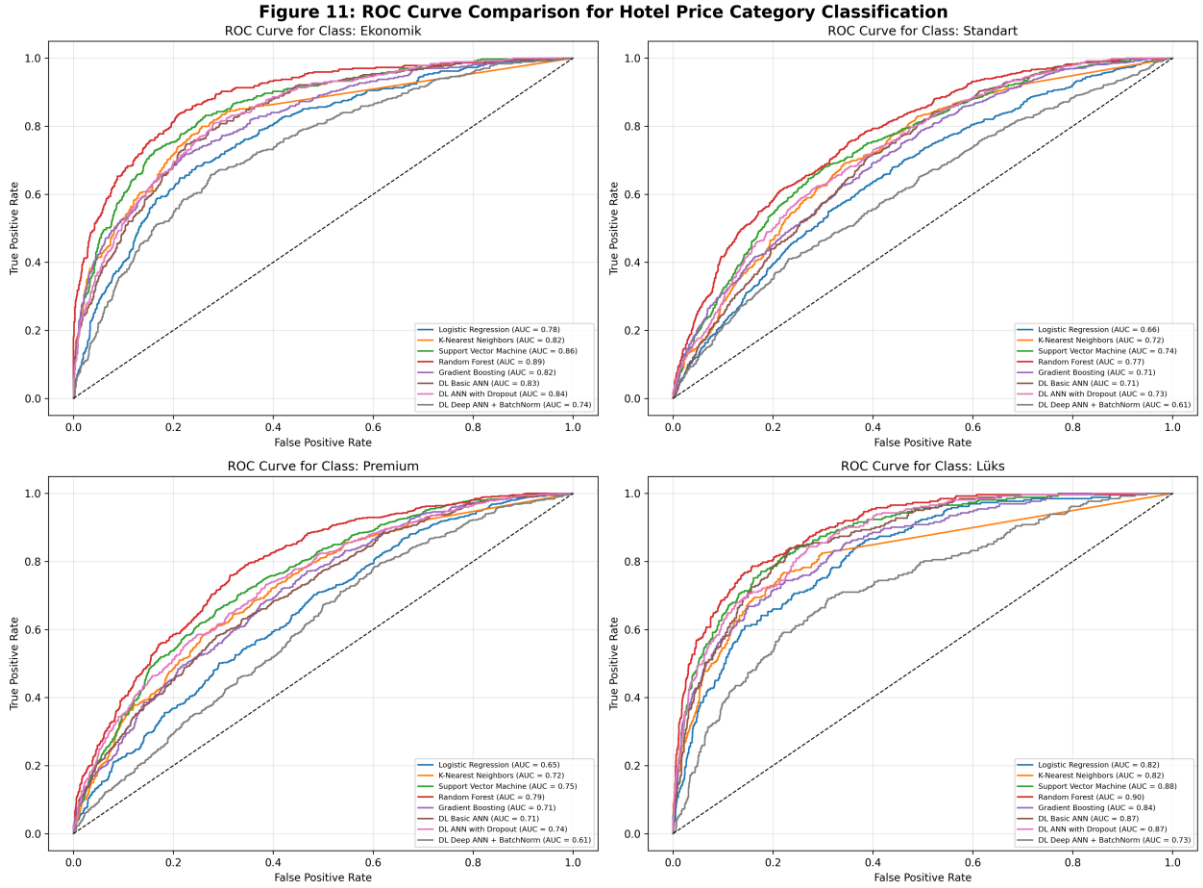


Figure 11: ROC curve comparison for hotel price category classification.

Random Forest achieved the highest AUC value of 83.66%. SVM followed with an AUC of 80.68%, and ANN with Dropout achieved an AUC of 78.89%. These results show that Random Forest had the strongest class separation capability.

Although the accuracy values were moderate, the AUC scores indicate that the models, especially Random Forest and SVM, learned meaningful patterns from the hotel dataset.

5.3 Confusion Matrix Analysis

Confusion matrices were used to examine how each model classified the four price categories. The classes were Economy, Standard, Premium, and Luxury.

The confusion matrices of all machine learning and deep learning models are presented in Figure 12.



Figure 12: Confusion Matrices for All Machine Learning and Deep Learning Models

The confusion matrices showed that many errors occurred between neighboring price categories. For example, some Standard hotels were confused with Premium hotels, and some Premium hotels were confused with Luxury hotels. This is expected because hotels near the category boundaries may have similar characteristics.

Random Forest produced the most balanced confusion matrix among the evaluated models. It classified the four categories more consistently compared to Logistic Regression and Deep ANN with Batch Normalization.

SMOTE was applied to the best accuracy model in order to analyze the effect of data balancing on real hotel data. According to the initial classification results, Random Forest achieved the best accuracy among the evaluated models; therefore, it was selected for the SMOTE-based comparison. SMOTE was applied only to the training data, while the test data was kept unchanged to avoid data leakage and to evaluate the model on real unseen records.

As shown in Figure 13, the Random Forest model was compared before and after applying SMOTE. Before SMOTE, the model correctly predicted 216 Economy, 471 Standard, 405 Premium, and 135 Luxury samples. After SMOTE, the correct predictions changed to 224 Economy, 451 Standard, 391 Premium, and 136 Luxury samples. These results show that SMOTE slightly improved the correct prediction count for the Economy and Luxury classes. However, the correct predictions for Standard and Premium classes slightly decreased.

Overall, SMOTE changed the decision boundaries of the Random Forest model by increasing the representation of minority classes in the training data. Although SMOTE did not improve every class equally, it provided a useful comparison for understanding how data balancing affects the prediction behavior of the best-performing model.

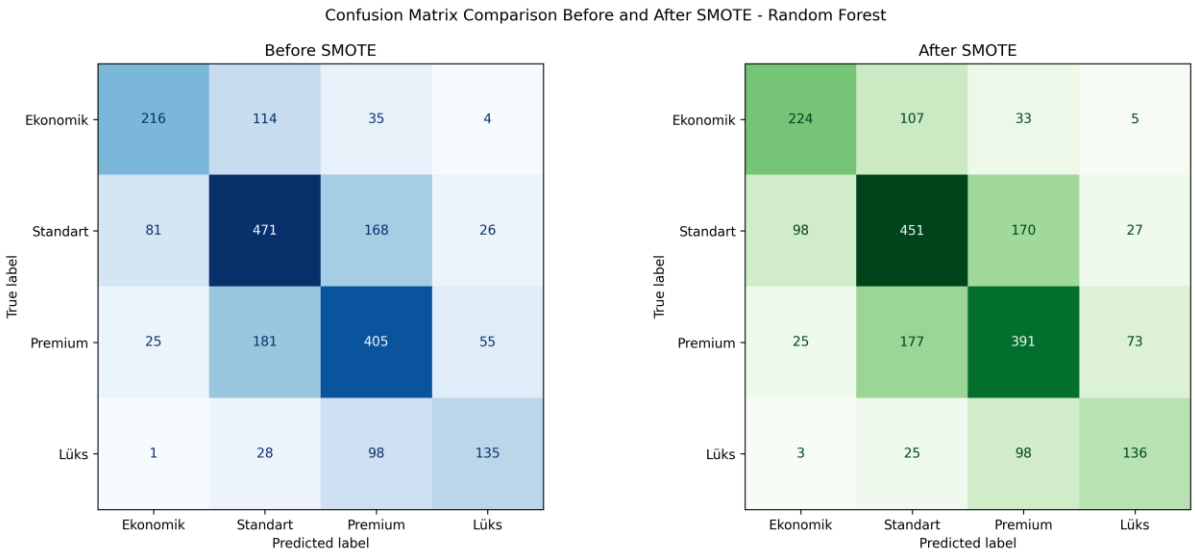


Figure 13: Confusion matrix comparison before and after applying SMOTE using Random Forest.

5.4 Training Time and Runtime Analysis

Training time and runtime for one record were measured to evaluate computational efficiency. The results are presented in Table 8.

Training Time and Runtime Comparison

Model	Training Time (sec)	Runtime for One Record (sec)
K-Nearest Neighbors	0.0066	0.0002692
Random Forest	3.8544	7.942e-05
Logistic Regression	9.3507	6.2e-07
DL Deep ANN + BatchNorm	14.1148	0.0001825
Gradient Boosting	19.7688	1.218e-05
DL Basic ANN	23.5661	0.00012312
DL ANN with Dropout	27.8451	0.00013348
Support Vector Machine	62.9173	0.00106142

Table 8: Training time and runtime comparison of classification models.

The fastest model during training was K-Nearest Neighbors with 0.0066 seconds. This is expected because KNN does not build a complex training model during the training phase. However, its runtime for one record was higher than Logistic Regression and Gradient Boosting.

Logistic Regression had the shortest runtime for one record with 6.2e-07 seconds, which shows that it is very efficient during prediction. Gradient Boosting also achieved a very low runtime for one record with 1.218e-05 seconds. On the other hand, Support Vector Machine had the longest training time with 62.9173 seconds and also one of the highest runtimes for one record with 0.00106142 seconds.

Random Forest achieved the best balance between classification performance and computational efficiency. Although it was not the fastest model during training, it achieved the best overall classification performance while maintaining a low runtime of 7.942e-05 seconds per record. Therefore, Random Forest was considered suitable for real-time hotel price category prediction tasks.

.

5.5 Feature Selection Results

Feature selection was performed using Random Forest feature importance scores. Since one-hot encoding was applied to categorical variables, feature selection was performed on the processed feature matrix.

The all-feature model used 111 processed features. The selected-feature model used the top 15 most important features.

The comparison between the all-feature Random Forest model and the selected-feature Random Forest model is presented in Table 9.

Feature Selection Performance Comparison

Model	Feature Count	Accuracy (%)	Precision (%)	Recall (%)	F1-score (%)	AUC (%)	Training Time (sec)	Runtime One Record (sec)
Random Forest - All Features	111	60.06	61.23	58.50	59.66	83.66	4.6835	0.00007665
Random Forest - Selected Features	15	52.77	53.47	48.82	50.37	78.27	6.8237	0.00017979

Table 9: Performance comparison of Random Forest using all features and selected features.

Although feature selection reduced the number of features from 111 to 15, the model performance decreased. This indicates that some of the removed one-hot encoded features still contained useful information for hotel price category classification.

Therefore, the all-feature Random Forest model was more successful for this dataset. However, the selected-feature model may still be useful when a simpler and lower-dimensional model is preferred.

5.6 Regression-Based Price Prediction Results

In addition to classification, a regression model was developed for the final web application. The purpose of this model was to predict the numerical hotel price first and then convert it into a category.

For this task, Random Forest Regressor was trained using `price_tl` as the target variable. A logarithmic transformation was applied to the target variable using `log1p`. After prediction, the result was converted back into Turkish Lira using `expm1`.

The Random Forest Regressor achieved an MAE of 11,180.66 TL, RMSE of 17,913.12 TL, and R^2 score of 0.482.

The MAE value shows that the model's average absolute prediction error was approximately 11,180.66 TL. The RMSE value was 17,913.12 TL, which indicates that larger prediction errors were also present in some cases. The R^2 score of 0.482 means that the model explained approximately 48.2% of the variance in hotel prices.

Although the regression model does not explain all price variation, this result is acceptable for a demo-based hotel price prediction system because hotel prices may be affected by additional external factors that are not included in the dataset. These factors may include brand reputation, booking date, local events, room type, demand level, discounts, and platform-specific pricing policies.

The regression model was selected for the final web application because it provides a more informative output than direct category prediction. Instead of only showing a category, the system first displays the predicted price and then assigns a category according to predefined price ranges.

6. Web Application Development

A Streamlit-based web application was developed to make the proposed hotel price prediction system user-friendly and accessible. The application allows users to enter hotel-related information through an interactive interface.

The user inputs include:country, city, season, hotel score, review count, distance to city center, star rating, hotel amenities

After the user enters the information, the application creates an input DataFrame. Then, the saved preprocessing pipeline transforms the input using the same preprocessing steps used during training. These steps include missing value handling, one-hot encoding, and numerical scaling.

After preprocessing, the trained Random Forest Regressor predicts the numerical hotel price. The predicted price is then converted into one of the four categories: Economy, Standard, Premium, Luxury.

The developed Streamlit web application interface is shown in Figure 14.

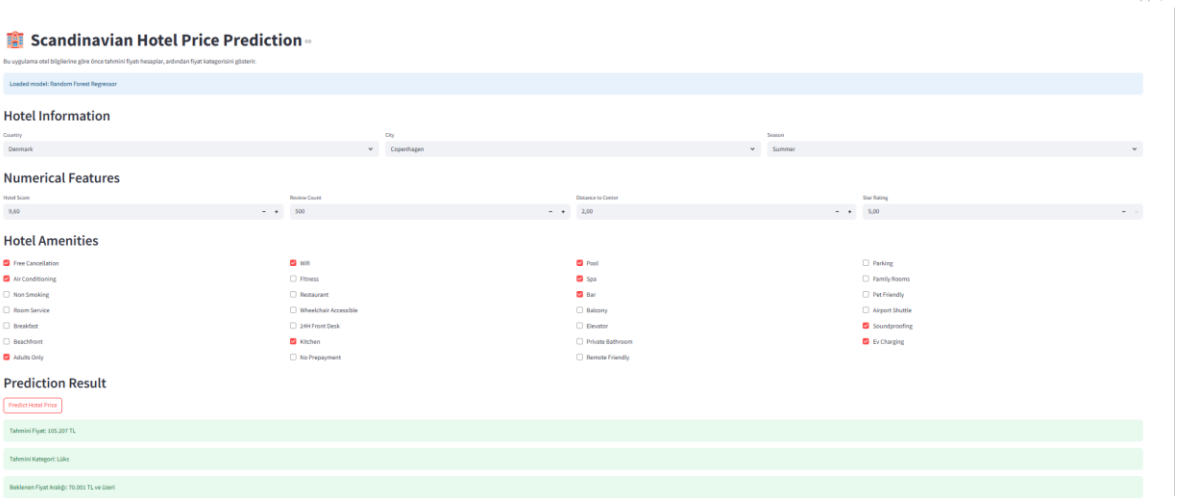


Figure 14: Streamlit web application interface for hotel price prediction.

This design provides a practical and understandable output for users. The system does not only classify the hotel but also gives a numerical price estimation. Therefore, the web application is more useful for real-world decision support.

The project folder was organized with separate components for the application file, dataset, saved model files, preprocessing pipeline, figures, tables, and requirements file. This structure makes the project easier to run, maintain, and demonstrate.

7. Overall Discussion

The results show that hotel price prediction is a challenging task because hotel prices are affected by many factors at the same time. Location, season, hotel score, review count, distance to center, star rating, and amenities all influence hotel prices.

Among the classification models, Random Forest achieved the best performance. This shows that ensemble-based tree models are suitable for structured tabular hotel data. The strong AUC value of Random Forest also indicates that the model was able to separate hotel price categories better than the other models.

Deep learning models did not outperform Random Forest. ANN with Dropout achieved the best deep learning result, but its F1-score was lower than Random Forest. This suggests that deep learning is not always superior for tabular datasets, especially when the dataset size is limited or when tree-based methods can capture feature interactions effectively.

Feature selection reduced the number of features significantly, but it also reduced performance. Therefore, using all processed features was more effective for this dataset. This result shows that one-hot encoded location and season features contributed useful information to the prediction task.

The regression model achieved an R^2 score of 0.482. This means that the model captured a meaningful part of the price variation, but there is still unexplained variation. This is reasonable because hotel prices may depend on external factors not included in the dataset.

Overall, the project successfully developed a complete machine learning pipeline, including own dataset collection through scraping, data cleaning, feature engineering, classification, regression, evaluation, feature selection, and web application deployment.

8. Conclusion

In this study, a machine learning-based hotel price prediction and classification system was developed using an original Scandinavian hotel dataset collected from the Booking application through scraping. The dataset was cleaned and transformed into a suitable format for machine learning. Hotel prices were converted into numerical Turkish Lira values, and price categories were created as Economy, Standard, Premium, and Luxury.

Five machine learning models and three deep learning models were trained and evaluated for hotel price category classification. Among these models, Random Forest achieved the best classification performance with 60.06% accuracy, 59.66% F1-score, and 83.66% AUC. The model also provided a low runtime for one record, making it suitable for real-time applications.

Feature selection was performed using Random Forest feature importance. The results showed that using all 111 processed features provided better performance than using only the top 15 selected features.

For the final web application, a Random Forest Regressor was trained to predict the numerical hotel price. The regression model achieved a MAE of 11,180.66 TL, RMSE of 17,913.12 TL, and R^2 score of 0.482. The model was integrated into a Streamlit web application that predicts the hotel price and then displays the corresponding price category and expected price range.

The developed system demonstrates how machine learning can be used to build an end-to-end hotel price prediction application. Future improvements may include collecting a larger dataset, adding room type information, using real-time booking data, applying hyperparameter optimization, and integrating live hotel pricing APIs.

References

[1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, pp. 5–32, 2001.

[3] J. H. Friedman, “Greedy Function Approximation: A Gradient Boosting Machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.

[4] C. Cortes and V. Vapnik, “Support-Vector Networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995.

[5] M. Abadi et al., “TensorFlow: Large-scale machine learning on heterogeneous systems,” 2015.

[6] Streamlit Inc., “Streamlit Documentation,” 2024.

[7] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016.

[8] G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *Advances in Neural Information Processing Systems*, 2017.

[9] Booking.com, “Booking.com Official Website,” Booking.com. [Online]. Available: https://www.booking.com/.